Schnorr Zero-Knowledge Authentication: Interactive vs. Fiat–Shamir

Jialiang Cheng
20JC150@queensu.ca
Queen's University
Kingston, ON, Canada

Abstract

This report presents a practical comparison of two Schnorr-based zero-knowledge authentication modes: an interactive protocol and a Fiat–Shamir non-interactive variant. Both modes allow a client to prove knowledge of a secret key without revealing that key, but they differ in challenge generation, message rounds, and operational constraints. We implement both flows in a shared codebase and evaluate them under a common benchmark pipeline. The evaluation focuses on authentication latency, communication cost, round trips, and replay-resistance behavior. In local experiments, the Fiat–Shamir mode achieves fewer round trips and modestly lower runtime and communication overhead. The findings show that the cryptographic core remains the same across modes, while the primary trade-off lies between interaction overhead and stricter engineering requirements for challenge binding and nonce lifecycle management.

1 Introduction and Topic Description

Zero-knowledge authentication is valuable because it enables trust without secret disclosure. In practical systems, this distinction directly affects incident impact: leaked logs, backup exposure, or service misconfiguration should not expose long-term client secrets.

This project examines one focused question: if the Schnorr verification equation is fixed but the challenge mechanism is changed, how does system behavior change in practice? We compare interactive Schnorr and Fiat–Shamir Schnorr within one implementation, one benchmark framework, and one threat-oriented test suite.

The protocol difference is narrow but consequential:

- Interactive mode generates challenge c online at the verifier.
- Fiat–Shamir mode derives c by hashing bound context.

All other variables are intentionally controlled to isolate the effect of interaction pattern. The comparison emphasizes four operational dimensions:

- average authentication latency,
- communication bytes per login,
- number of round trips,
- replay resistance under one-time state controls.

2 Background

2.1 Interactive Schnorr Protocol

Let p and q be primes, let $G \subseteq \mathbb{Z}_p^*$ be the order-q subgroup generated by g, let $x \in \mathbb{Z}_q$ be the secret key, and let $y = g^x \bmod p$ be the public key. For one authentication attempt, the prover samples $r \in \mathbb{Z}_q$, sends commitment $t = g^r \bmod p$, receives challenge $c \in \mathbb{Z}_q$, and returns:

$$s = r + cx \pmod{q}.$$

The verifier accepts if:

$$g^s \equiv t \cdot y^c \pmod{p}.$$

This flow is straightforward to audit because freshness is explicit: the verifier generates c for each session.

2.2 Fiat–Shamir Non-Interactive Variant

Fiat–Shamir removes online challenge generation and replaces it with hash-based derivation [2]. In this implementation, challenge generation is defined by

$$c = \mathrm{int}(\mathrm{SHA256}(\mathrm{user_id} \| \mathrm{login_nonce} \| y \| t)) \bmod q,$$

where $\mathrm{int}(\cdot)$ converts the digest bytes to a nonnegative integer and $\|$ denotes canonical byte-string concatenation. The prover sends $(t, c, s, \mathrm{login_nonce})$, and the verifier recomputes the same challenge value before checking the Schnorr verification equation.

The algebraic verification equation remains unchanged; only the protocol orchestration layer changes. This distinction drives the comparative analysis.

2.3 Zero-Knowledge Properties in This Project

The implementation is evaluated with respect to the three standard properties of identification-style zero-knowledge proofs. **Completeness** is observed when an honest prover is accepted in both modes. **Soundness intuition** is supported by rejection of modified or forged responses in the test suite. **Zero-knowledge intuition** is maintained because the server receives proof transcripts but never the secret key.

2.4 Why the Difference Matters in Practice

In local testing, a reduction of one round may appear minor. In real deployments, it is often decisive. For example, on unstable mobile links, each additional request-response step increases timeout risk and retry pressure. Reducing one round can therefore improve reliability as much as it improves speed.

However, reduced interaction introduces stricter state-governance requirements. Challenge binding must be complete, and nonce usage must be one-time. If a deployment permits nonce reuse across workers, replay resistance degrades even if the core cryptography is implemented correctly.

3 Discussion of Selected Papers

3.1 Paper Summaries

Schnorr foundation [1]. Schnorr provides an efficient identification protocol with compact proofs and low verifier cost. Its structure is

simple enough for implementation and strong enough for rigorous security reasoning.

Fiat–Shamir transformation [2]. Fiat and Shamir show how to replace verifier-generated randomness with a hash challenge, transforming interactive identification into a non-interactive proof form (under the random-oracle methodology) while preserving the same core relation.

Modern cryptographic perspective [3]. Contemporary references emphasize that deployment security depends not only on formal reductions but also on encoding discipline, domain separation, and state lifecycle design.

3.2 Technical Commentary and Critique

Interactive Schnorr provides a clearer analytical baseline for auditing and incident analysis. Freshness is explicit, session progression is visible, and verifier control is direct.

Fiat–Shamir is advantageous when latency budgets are tight. The benchmark supports this claim. Yet the gain is conditional on strong implementation discipline: every challenge-input field must be stable, authenticated, and encoded consistently.

A concrete example illustrates the risk. Suppose one service instance includes `user_id` in the hash input while another omits it because of a serialization bug. The resulting challenge mismatch creates intermittent failures; under adverse conditions, this inconsistency can also create replay-related failure modes that are difficult to diagnose.

The comparison should therefore be interpreted at two layers. Cryptographically, the two modes are closely related. Operationally, they impose different reliability and governance contracts.

3.3 Relationship Between the Two Approaches

The non-interactive mode is a transformation of the interactive baseline, not an unrelated construction. Our implementation follows this dependency in sequence:

- establish a correct interactive flow and tests,
- replace verifier-issued challenge with hash-derived challenge,
- preserve the same proof equation and verification arithmetic,
- compare only the orchestration delta.

This design supports a clear causal interpretation: the observed differences arise from interaction patterns and state semantics rather than from a change in the underlying cryptographic primitive.

3.4 Comparison of Limitations

Interactive limitations. More round trips, more session transitions, and greater sensitivity to network-induced tail latency.

Fiat–Shamir limitations. Fewer messages, but stricter requirements for nonce uniqueness, challenge-input completeness, and deterministic encoding.

Shared limitations. This work is a teaching-oriented prototype and does not include production-grade transport hardening, distributed state coordination, or full observability infrastructure.

4 Implementation and Evaluation

4.1 Code Structure

The implementation components are summarized as follows:

- `src/zk_auth/schnorr.py`: group operations, proof generation, verification, and subgroup checks.
- `src/zk_auth/app.py`: registration, interactive and Fiat–Shamir login flows, and one-time state handling.
- `demo.py`: single-session demonstration.
- `benchmark.py`: repeated trials and CSV export.
- `results/benchmark.csv`: benchmark output used in the report tables.
- `tests/test_auth.py`: correctness, forgery rejection, replay rejection, and input validation.

4.2 Resources, Software, and Security Parameters

The implementation is written in Python 3 and uses standard-library components for randomness and SHA-256-based challenge derivation [4, 6]. Testing uses pytest [5]. No dedicated zero-knowledge library is used in the proof core; this improves instructional transparency but is a project limitation. The demo parameters are approximately 128-bit for p and 127-bit for q with $p = 2q + 1$, which are suitable for reproducible coursework but not for production security. Project repository: https://github.com/DracoCheng/cisc878-zk-auth.

4.3 Threat Model and Assumptions

The evaluation considers replay-capable network adversaries and malformed-input attempts. Transport confidentiality is not assumed within this prototype; replay protection is therefore enforced at the protocol-state layer.

The threat model does not include side-channel attacks, hardware compromise, or cross-service trust collapse. These threats are important in production settings but are out of scope for this project.

Critical assumptions are:

- strong randomness for prover nonces,
- collision resistance of the hash function used for challenge derivation,
- strict one-time semantics for interactive sessions and server-issued login nonces.

4.4 Methodology

Evaluation uses a controlled repeated-trial design in which the execution environment is fixed and only the protocol mode is varied. Three primary metrics are reported: average authentication latency (ms), average communication cost (bytes), and required round trips per successful login.

To strengthen validity, performance measurements are interpreted together with behavioral security checks, including forgery rejection and replay-resistance verification under one-time state policies. Benchmarks were run locally on Apple M3 (8 GB), macOS 26.4, and Python 3.13; absolute timing values are therefore environment-dependent.

4.5 Results

Local run (April 4, 2026):

Table 1 reports absolute benchmark values, and Table 2 reports relative differences with interactive Schnorr as the baseline. The primary effect is the reduction from two rounds to one in Fiat–Shamir mode, accompanied by decreases in latency and communication overhead.

Table 1: Interactive vs. Fiat–Shamir benchmark results

Mode	Avg time (ms)	Avg bytes	Round trips
Interactive	0.1586	328.15	2
Fiat–Shamir	0.1348	300.93	1

Table 2: Relative change (Fiat–Shamir vs. Interactive)

Metric	Change
Avg time	−15.01%
Avg bytes	−8.29%
Round trips	−50.00%

4.6 Layered Analysis

Layer 1: protocol mechanics. Fiat–Shamir halves round trips, which explains most of the observed gain.

Layer 2: communication profile. The byte reduction is moderate rather than dramatic because both modes still transmit commitment and response material; only challenge handling differs.

Layer 3: runtime behavior. Runtime improvements are measurable but bounded. Both modes execute similar modular arithmetic, so CPU-side differences remain limited in local runs.

Layer 4: operational meaning. In production networks, saving one round often has a disproportionate effect on p95 and p99 authentication latency. This observation gives stronger deployment significance to the result than raw millisecond deltas alone.

4.7 Security Discussion with Concrete Attack Scenarios

Scenario A: transcript replay without one-time controls. An attacker captures a successful transcript and resubmits it. Interactive mode rejects this attack when session IDs are single-use and challenge state is consumed. Fiat–Shamir rejects it only when server nonce usage is strictly one-time. The example demonstrates that replay resistance is a state-machine property, not merely an algebraic one.

Scenario B: malformed public-key injection. If the verifier accepts keys outside the intended subgroup, protocol assumptions can be violated through crafted edge cases. The implementation rejects malformed subgroup elements before authentication begins; this validation is part of the security boundary.

Scenario C: inconsistent hash-input encoding. If different service instances serialize challenge inputs in different field orders (for example, placing the user identifier before the nonce in one instance but not in another), honest clients may fail unpredictably and operators lose diagnostic clarity. Canonical encoding is therefore mandatory in non-interactive deployments.

5 Concluding Remarks and Future Directions

5.1 Primary Takeaways

The comparison supports a clear conclusion. Interactive Schnorr is easier to audit and reason about, whereas Fiat–Shamir is more efficient in end-to-end interaction. The appropriate choice depends on which control surface an organization can enforce more reliably: session choreography or nonce/binding governance.

The data and tests support three practical recommendations:

- choose interactive mode when verifier-controlled freshness and debugging transparency are primary objectives,
- choose Fiat–Shamir when latency and round-trip reduction are primary objectives,
- in either mode, treat subgroup validation and one-time state semantics as non-negotiable controls.

5.2 Open Challenges

- evaluate both modes under WAN latency, packet loss, and concurrent load,
- introduce persistent state backends and test crash-recovery invariants,
- benchmark larger parameter sets and long-running workloads,
- instrument explicit alerts for nonce reuse and replay-window anomalies.

5.3 Future Research Directions

A practical next step is adaptive mode selection: retain interactive freshness control for tightly managed internal services and prefer Fiat–Shamir at latency-sensitive edges. A second direction is formal verification of session and nonce state machines so replay guarantees remain robust during scaling and incident response.

5.4 Vision for the Next Few Years

In the near term, this project can serve as a reusable evaluation template for privacy-preserving authentication modules in coursework and early-stage systems. In the medium term, zero-knowledge login is likely to become a standard framework option with explicit policy controls for interaction level, latency budget, and replay-hardening strictness.

References

[1] C. P. Schnorr. 1991. Efficient signature generation by smart cards. *Journal of Cryptology* 4, 3 (1991), 161–174.

[2] A. Fiat and A. Shamir. 1987. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology—CRYPTO '86*. 186–194.

[3] J. Katz and Y. Lindell. 2020. *Introduction to Modern Cryptography* (3rd ed.). CRC Press.

[4] Python Software Foundation. n.d. Python 3 Documentation. Accessed April 5, 2026. https://docs.python.org/3/.

[5] pytest Development Team. n.d. pytest Documentation. Accessed April 5, 2026. https://docs.pytest.org/.

[6] National Institute of Standards and Technology (NIST). 2015. Secure Hash Standard (SHS), FIPS PUB 180-4. https://doi.org/10.6028/NIST.FIPS.180-4.